Notes on On-Policy Monte Carlo Control

1 Setting

We work in the usual MDP framework: an agent interacts with an environment through states $s \in \mathcal{S}$, actions $a \in \mathcal{A}$, rewards $r \in \mathbb{R}$, and a discount factor $\gamma \in [0, 1]$. A policy $\pi(a \mid s)$ is a (possibly stochastic) mapping from states to actions. The action-value function under π is

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k} \,\middle|\, S_t = s, A_t = a \right].$$

Monte Carlo (MC) methods estimate Q^π directly from *complete episodes* of experience, without requiring a model of the environment's dynamics (unlike dynamic programming). *On-policy* means the policy used to generate the behaviour (and therefore the data) is the same policy we are trying to evaluate and improve.

2 Why not just be greedy?

If we always act greedily with respect to the current estimate $\hat{Q}(s, a)$, we may never visit state-action pairs that look bad only because they have been sampled too few times. MC control needs *maintained exploration*. The standard fix is an ε-greedy policy:

$$\pi(a \mid s) = \begin{cases} 1 - \varepsilon + \dfrac{\varepsilon}{|\mathcal{A}(s)|}, & a = \arg\max_{a'} \hat{Q}(s, a') \\ \dfrac{\varepsilon}{|\mathcal{A}(s)|}, & \text{otherwise.} \end{cases}$$

This keeps every action selectable with probability at least $\varepsilon/|\mathcal{A}(s)|$, so every (s, a) pair keeps getting visited in the limit.

2.1 GLIE: Greedy in the Limit with Infinite Exploration

For MC control to provably converge to Q^*, the policy sequence $\{\pi_k\}$ should satisfy the **GLIE** conditions:

(i) Every state-action pair is visited infinitely often.

(ii) The policy converges to a greedy policy in the limit, e.g. $\varepsilon_k \to 0$ as $k \to \infty$ (a common choice is $\varepsilon_k = 1/k$).

In practice, many implementations (including the one in this notebook) use a small *fixed* ε instead of decaying it. This sacrifices the formal convergence guarantee but is simpler and works well enough for small, stochastic-free-ish problems: the learned policy converges to a near-optimal ε-greedy policy rather than the exactly optimal greedy one.

3 Generalized Policy Iteration with MC

MC control instantiates **Generalized Policy Iteration (GPI)**: alternate between

- **Policy evaluation**: estimate $Q^\pi(s,a)$ from sampled returns.
- **Policy improvement**: make π (ε-)greedy with respect to the current $\hat{Q}$.

Unlike DP, evaluation here is done from *sampled episodes* rather than full sweeps over the state space, and the two steps are interleaved at the granularity of a single episode rather than run to convergence separately.

4 Computing returns from an episode

Given an episode $S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_{T-1}, A_{T-1}, R_T$, the return from time t is

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}.$$

It is computed efficiently backward through the trajectory:

$$G_T = 0, \qquad G_t = R_{t+1} + \gamma\, G_{t+1}.$$

5 Every-visit MC control (sample-average updates)

Let Returns(s,a) be the (growing) list of returns observed after visiting (s,a). The value estimate is the sample mean:

$$\hat{Q}(s,a) = \frac{1}{N(s,a)} \sum_{i=1}^{N(s,a)} G^{(i)}(s,a).$$

Algorithm 1 On-policy every-visit MC control (ε-greedy, sample average)

Initialize $\hat{Q}(s,a) \leftarrow 0$ for all s, a; Returns$(s,a) \leftarrow \emptyset$
for episode $= 1, 2, \ldots$ **do**
  Generate episode $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$ using ε-greedy w.r.t. $\hat{Q}$
  $G \leftarrow 0$
  for $t = T-1, T-2, \ldots, 0$ **do**
    $G \leftarrow R_{t+1} + \gamma G$
    append G to Returns(S_t, A_t)
    $\hat{Q}(S_t, A_t) \leftarrow \text{mean}\big(\text{Returns}(S_t, A_t)\big)$
  end for
end for

Note this is *every-visit*, not first-visit: if (S_t, A_t) repeats within the same episode, each occurrence contributes its own return to the average. This is exactly what `on_policy_mc_control` does in `Section_4_on_policy_control_complete.ipynb`: it stores all returns per (s,a) pair in a dictionary and recomputes `np.mean` after every episode.

A useful incremental identity avoids storing the whole return history: with $N \leftarrow N + 1$,

$$\hat{Q}(s,a) \leftarrow \hat{Q}(s,a) + \frac{1}{N(s,a)}\Big(G - \hat{Q}(s,a)\Big).$$

6 Constant-α MC control

Replacing the diminishing step size $1/N(s,a)$ with a fixed $\alpha \in (0,1]$ gives **constant-α MC**:

$$\hat{Q}(s,a) \leftarrow \hat{Q}(s,a) + \alpha\Big(G - \hat{Q}(s,a)\Big).$$

Algorithm 2 On-policy MC control, constant step size α

Initialize $\hat{Q}(s,a) \leftarrow 0$ for all s, a
for episode $= 1, 2, \ldots$ **do**
    Generate episode using ε-greedy w.r.t. $\hat{Q}$
    $G \leftarrow 0$
    for $t = T-1, T-2, \ldots, 0$ **do**
        $G \leftarrow R_{t+1} + \gamma G$
        $\hat{Q}(S_t, A_t) \leftarrow \hat{Q}(S_t, A_t) + \alpha\big(G - \hat{Q}(S_t, A_t)\big)$
    end for
end for

This is the `constant_alpha_mc` function in `opmc_c/Section_4_on_policy_constant_alpha_mc_complete.ipynb`

Why use constant α? Two practical reasons:

- **Non-stationarity.** Because the policy keeps changing (it is greedy w.r.t. a moving $\hat{Q}$), older returns were generated under a stale, worse policy. A sample average weighs all past returns equally; constant-α instead forms an exponentially decaying weighted average, giving recent returns — generated under a more recent, better policy — more influence.
- **Memory.** No need to store the full list of past returns per (s,a), just the running estimate.

The trade-off is that the estimate no longer converges to the true mean (it keeps tracking / fluctuating), so α acts as a bias-variance / adaptivity knob: larger α tracks a moving target faster but is noisier; smaller α is smoother but slower to adapt.

7 First-visit vs. every-visit

- **First-visit MC**: only the *first* occurrence of (s,a) in an episode contributes a return to the average.
- **Every-visit MC**: *every* occurrence contributes.

Both converge to Q^π under standard assumptions (each is an unbiased or asymptotically consistent estimator), but every-visit is simpler to implement — no need to track "have we seen this pair yet this episode?" — which is why the notebook implementation above uses it.

8 Convergence remarks

- MC control with GLIE exploration and sample-average updates converges to the optimal Q^* (and hence an optimal ε-greedy-turned-greedy policy) with probability 1, as both the number of episodes and the exploration schedule satisfy the conditions above.
- With *fixed* ε and/or constant α, the method instead converges (in a suitable stochastic-approximation sense) to a stationary distribution around a near-optimal ε-greedy policy, trading exact optimality for simplicity, adaptivity to non-stationarity, and faster empirical learning.
- MC methods require *episodic* tasks (no bootstrapping): the return G_t can only be computed once the episode terminates. This is the key difference from TD-based control methods (Sarsa, Q-learning), which bootstrap from $\hat{Q}$ and can learn online, step by step, in continuing tasks.

9 Reference

Sutton, R. S., & Barto, A. G. *Reinforcement Learning: An Introduction*, 2nd ed., Chapter 5 (Monte Carlo Methods).